

HEARSPACE

서비스 전체 플로우와 구조 설계 보고서

3조 이지호 20201068 · 박상우 20214327 · 박민서 20226292

2026. 10. 09.

목차

1. 서비스 개요	1
2. 서비스 전체 플로우	2
3. 전체 서비스 구조 설계	5
4. 확장 계획	8

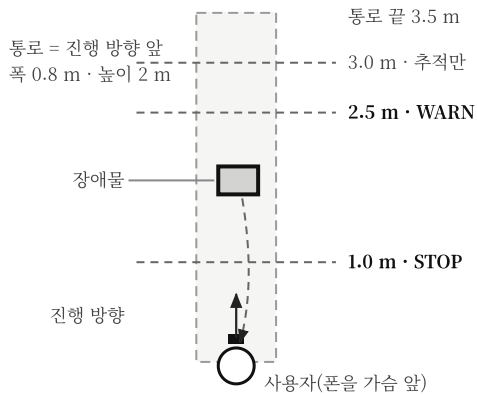
1 서비스 개요

사용자가 스마트폰을 가슴 앞에 들고 걸으면, 앱은 폰의 위치와 화면 각 픽셀까지의 거리를 읽어 **진행 방향 앞의 장애물**을 찾고, 그 장애물이 있는 **방향에서 들리는 소리**(공간음향)로 이어폰에 알린다. 소리의 좌우는 장애물의 방향을, 크기는 거리를, 음높이는 높이를 나타낸다. 서비스는 힌지팡이를 대체하지 않는 **보조 수단**이며, 지팡이가 닿기 전(약 3 m 앞)부터 장애물을 미리 알리는 것을 목표로 한다.

표 1. 구성과 입출력.

구분	내용
사용 기기	ARCore 깊이 기능(Depth API)을 지원하는 Android 스마트폰, 유선·무선 이어폰. 폰은 세로로 가슴 앞에 든다
입력	ARCore가 주는 폰의 위치·방향(자세), 깊이 영상(160 × 90), 카메라 내부 파라미터, 촬영 시각. 사용자 조작은 화면 두 번 탭·길게 누르기
출력	장애물 공간음향(최대 2개 동시), 상태 알림음(시작·준비 완료·정지·확인 불가·대기), 진동, 저시력용 고대비 화면(상태 글자·방향 화살표)

(a) 위에서 본 보행 장면



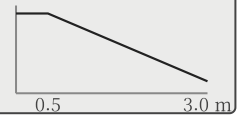
(b) 장애물 정보를 소리로 바꾸는 규칙

좌우 방향 → 소리가 오는 방향

머리 기준 방위각으로 두 귀의 소리를 다르게 만든다
(HRTF 바이노럴 렌더링)

거리 → 음량

0.5 m 이내 가장 크고 3.0 m에서 가장 작다



높이(머리 기준) → 음높이

-1 m 250 Hz · 0 m 500 Hz · +1 m 1000 Hz
거리가 바뀌어도 같은 높이면 같은 음

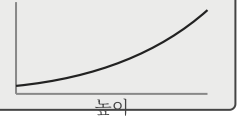


그림 1. 서비스 개념. (a) 진행 방향 앞의 통로 안에 들어온 장애물만 안내하고, 거리에 따라 STOP(1.0 m 안)·WARN(2.5 m 안)·추적만(3.0 m 안)으로 나눈다. (b) 장애물 하나의 방향·거리·높이를 서로 다른 소리 성질에 하나씩 대응시켜, 소리만 듣고 세 정보를 구분하게 한다.

2 서비스 전체 플로우

2.1 사용자 이용 흐름

사용자 모드는 화면을 보지 않고 쓰도록 **화면 전체를 버튼 하나로** 만들었다. 조작은 두 가지뿐이다. 두 번 탭은 시작·일시정지·재개를, 길게 누르기는 종료를 한다(TalkBack 사용 시 TalkBack의 같은 동작). 모든 상태 변화는 알림음과 진동으로 알리고, 바뀐 상태는 TalkBack이 읽는다.

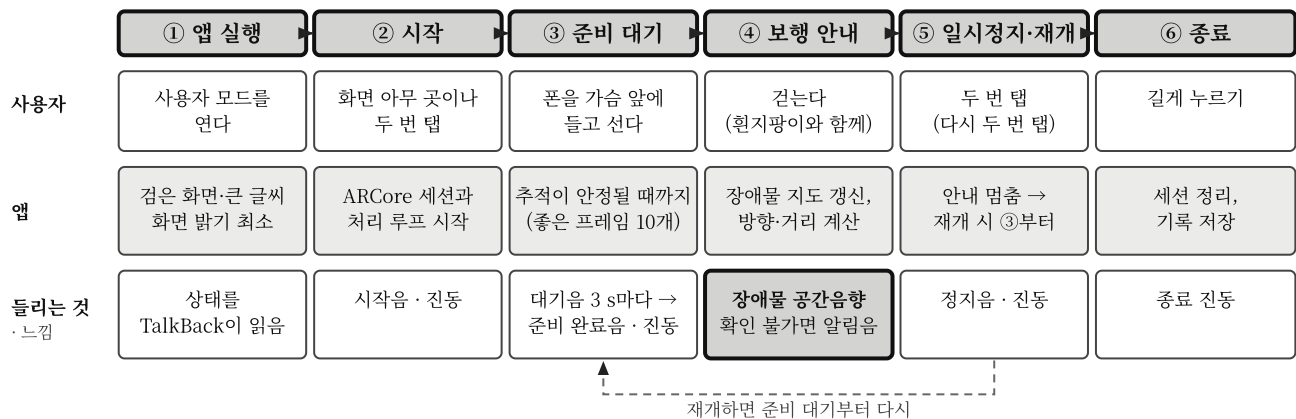


그림 2. 사용자 이용 흐름. 위에서부터 사용자의 행동, 앱의 처리, 사용자가 듣고 느끼는 것이다. 사용자는 걷는 동안(④) 아무 조작 없이 소리만 듣는다. 준비가 끝나기 전에는 장애물 안내를 하지 않고 대기음으로 “아직 준비 중”임을 알린다.

2.2 안내 상태 흐름

안내 중에도 위치 추적이 끊기거나 정보가 늦어질 수 있다. 이때 소리가 멈추면 사용자는 “앞에 아무것도 없다”로 오해할 수 있으므로, 앱은 **정보를 믿을 수 있는지를** 상태로 관리하고 믿을 수 없으면 침묵 대신 “확인 불가” 알림을 낸다.

지연 시간 = 지금 시각 - 안내에 쓰는 장애물 정보를 만든 깊이 영상의 촬영 시각

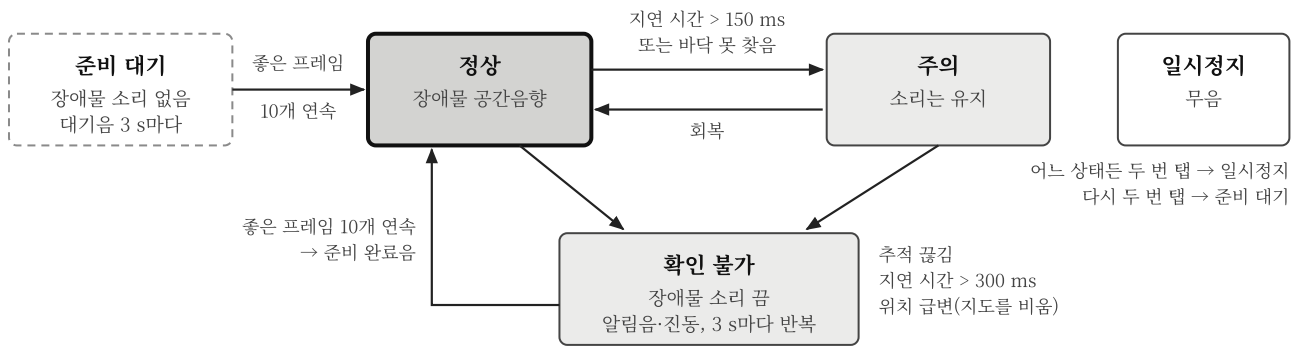


그림 3. 안내 상태 흐름. 지연 시간이 허용치(300 ms)의 절반을 넘거나 바닥을 찾지 못하면 “주의”로 두되 소리는 유지하고, 허용치를 넘거나 추적이 끊기면 장애물 소리를 끄고 “확인 불가”를 반복해 알린다. 확인 불가에서는 좋은 프레임이 10개 연속이어야 다시 안내한다. 폰 위치가 순간적으로 비현실적인 속도(15 m/s, 600°/s 초과)로 바뀌면 지도를 비운다.

2.3 한 순간의 처리 흐름: 센서에서 소리까지

처리는 두 속도로 나뉜다. 깊이 영상에서 장애물 지도를 만드는 일은 무거워(깊이 한 장에 수~수십 ms) 느린 경로가 깊이 영상이 올 때마다 하고, 소리의 방향은 고개나 몸을 돌리면 바로 따라와야 하므로 빠른 경로가 오디오 블록(5.3 ms)마다 최신 자세로 다시 계산한다. 장애물은 세상(월드 좌표)에 고정해 두므로 지도가 조금 늦게 갱신돼도 몸을 돌렸을 때의 방향은 틀리지 않는다.

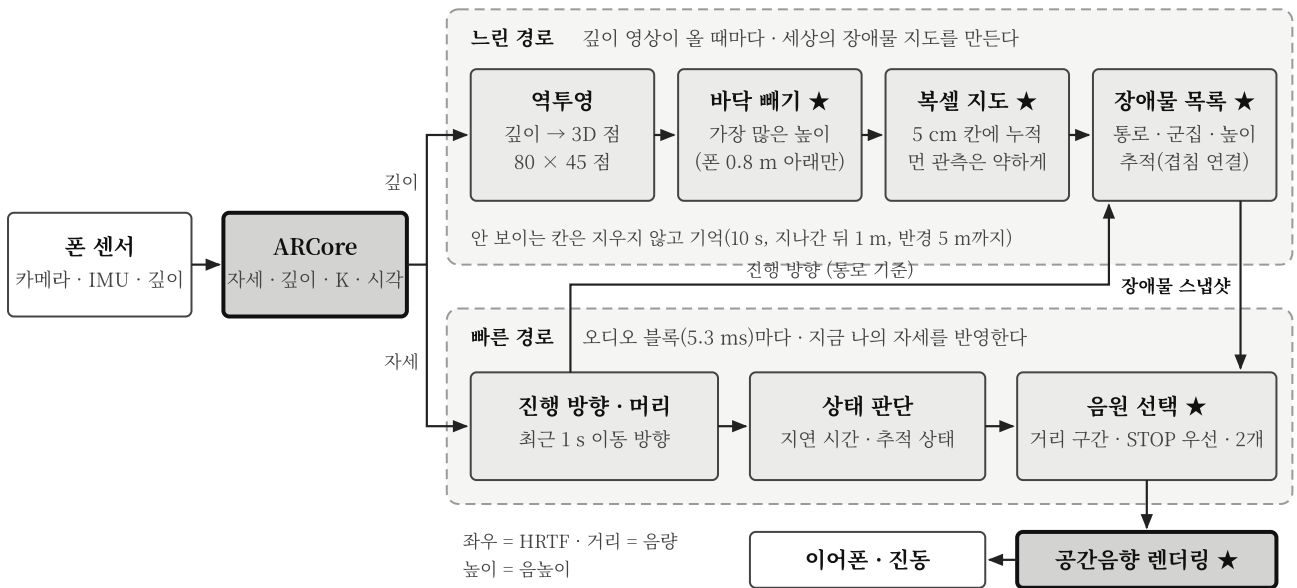


그림 4. 센서에서 소리까지의 처리 흐름. 굵은 테두리는 외부 라이브러리(ARCore)와 출력(공간음향), 회색 상자는 팀이 구현한 처리다. 두 경로는 “장애물 스냅샷”과 “진행 방향”만 주고받는다. ★는 성능 개선 단계(M12·M13, 공간음향 개선)에서 바뀌어 지금 기본값에 반영된 부분이다(3.5절).

표 2. 처리 단계별 입출력.

단계	입력 → 출력	하는 일
역투영	깊이 영상 → 월드 3D 점	픽셀마다 깊이와 카메라 내부 파라미터로 3D 점을 만들고, 깊이를 찍은 시각의 자세로 세상 좌표에 놓는다(2픽셀 간격, 80×45)
바닥 빼기	3D 점 → 바닥 높이, 바닥 아닌 점	폰보다 0.8 m 이상 아래인 점의 높이 분포에서 가장 많은 높이를 바닥으로 본다. 바닥 근처 점은 장애물에서 뺀다
복셀 지도	바닥 아닌 점 → 점유 칸	세상을 5 cm 칸으로 나눠 점이 맞은 횟수를 쌓는다. 2 m보다 먼 관측은 $(2/\text{거리})^2$ 만큼 약하게 센다. 비어 보인 칸은 점수를 깎고, 시야 밖 칸은 그대로 기억한다
장애물 목록	점유 칸 → 장애물 (위치·크기·높이 유형·추적 번호)	진행 방향 앞 통로(폭 0.8 m, 길이 3.5 m, 높이 2 m) 안 칸만 남기고 통로 가장자리의 긴 벽은 뺀 뒤, 가까운 칸끼리 묶는다(DBSCAN). 높이 유형(바닥에서 올라온 것·머리 높이 돌출)과 통로 안 가장 가까운 점(대표점)을 정하고, 이전 장애물과 이어 같은 번호를 준다
진행 방향·머리	자세 → 진행 방향, 머리 위치	폰을 돌리는 손목 방향이 아니라 최근 1 s 동안 몸이 이동한 방향 을 진행 방향으로 쓴다. 소리의 기준점은 폰이 아니라 머리(폰에서 위 0.5 m, 뒤 0.39 m)다
상태 판단	추적 상태·지연 시간 → 상태	그림 3의 상태를 정한다
음원 선택	장애물 → 음원 명령(최대 2개)	머리 기준 방향·거리를 다시 계산해 거리 구간을 정하고, 통로 안 STOP을 먼저, 그다음 가까운 순으로 고른다
공간음향 렌더링	음원 명령 → 좌우 오디오	그림 1(b)의 규칙으로 음량·음높이를 정하고, HRTF로 방향을 입힌다

2.4 언제 소리를 내는가

표 3. 거리 구간과 소리. 구간은 머리에서 진행 방향으로 잰 거리다. 덜 급한 구간으로 나갈 때만 0.15 m 여유를 두어, 경계에서 소리가 깜빡이지 않으면서도 가까워질 때는 늦지 않게 했다.

구간	거리	소리
STOP	1.0 m 안	통로 안이면 멈춰 있어도 계속 울리고, 다른 음원은 작게 줄인다(STOP이 묻히지 않게)
WARN	1.0~2.5 m	새 장애물이 나타나거나 구간이 바뀌면 0.7 s 울리고, 다가가는 동안 (초속 0.1 m 이상) 계속 울린다
추적만	2.5~3.0 m	소리 없이 추적한다(다가오면 바로 WARN으로 이어지게)
그 밖	3.0 m 밖, 통로 밖	안내하지 않는다. 지도에는 기억한다

멈춰 선 채 같은 WARN 장애물이 계속 울리면 피로하므로, 다가가지 않는 WARN은 한 번 알린 뒤 조용해진다. 이 무음은 “장애물 없음”이 아니라 “변화 없음”이며, 정보 자체를 믿을 수 없을 때는 항상 확인 불가 알림을 낸다.

3 전체 서비스 구조 설계

3.1 시스템 구성

시스템은 기기·플랫폼 위에 **앱 모듈(Android)**과 **코어 모듈(순수 Kotlin)** 두 층으로 나뉜다. 장애물 인식부터 음원화까지의 알고리즘은 모두 코어에 두어, 기기 없이 PC에서도 **같은 코드로** 녹화 세션을 재생하고 시험할 수 있다. 앱 모듈은 ARCore·오디오 장치·화면처럼 Android에만 있는 것을 말한다.

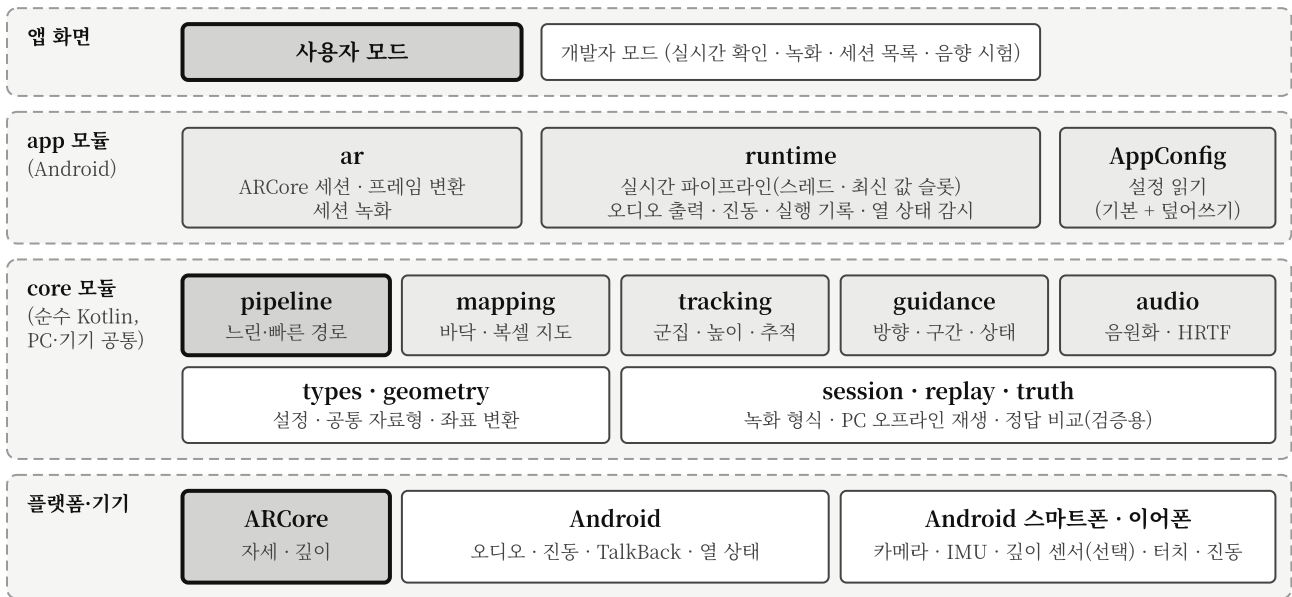


그림 5. 시스템 구성. 위 계층은 바로 아래 계층의 기능을 사용한다. 굵은 테두리는 사용자에게 보이는 화면, 핵심 실행 모듈(pipeline), 외부 라이브러리(ARCore)다. core는 Android를 전혀 쓰지 않아 PC 재생과 기기 실행이 같은 알고리즘으로 동작한다.

표 4. 모듈별 역할.

모듈	역할	대표 구성 요소
ui	사용자 모드(두 번 탭·길게 누르기, 고대비 화면, TalkBack), 개발자 모드(실시간 화면·녹화)	UserModeActivity, DevHomeActivity
ar	ARCore 세션 관리, 프레임을 코어 자료형(자세·깊이)으로 변환, 세션 녹화	ArSessionManager, FrameAdapter, Recorder
runtime	세 루프 실행과 최신 값 전달, 오디오 장치 출력, 진동, 실행 기록, 열 상태 기록	LivePipeline, BinauralOutput, Haptics, ThermalMonitor
pipeline	느린 경로(깊이 → 장애물 스냅샷), 빠른 경로(자세 + 스냅샷 → 음원 명령)	SlowPath, FastPath
mapping	바닥 높이 추정, 5 cm 복셀 지도(누적·감쇠·시야 밖 기억)	Floor, VoxelMap, LocalMap
tracking	군집, 높이 유형, 대표점, 장애물 추적(번호 유지)	Cluster, HeightClass, RepPoint, Tracker
guidance	진행 방향, 통로, 거리 구간·음원 선택, 안내 상태 기계	Heading, Corridor, Policy, StateMachine
audio	음원 명령을 소리로(거리·높이 매핑, 알림음), HRTF 바이노럴 렌더링	RiskSoundGenerator, BinauralRenderer, Hrtf

모듈	역할	대표 구성 요소
types	설정 읽기·검증, 공통 자료형(자세·깊이·장애물·음원 명령)	Config, ConfigLoader, Types

3.2 실행 구조: 세 루프와 최신 값 전달

앱은 세 루프를 동시에 돌린다. 루프 사이에는 줄을 세우는 큐 대신 **최신 값 하나만 담는 슬롯**을 둔다. 처리가 늦어지면 오래된 값을 쌓아 두었다가 뒤늦게 안내하는 대신, 지난 값을 버리고 가장 최근 값만 쓴다.

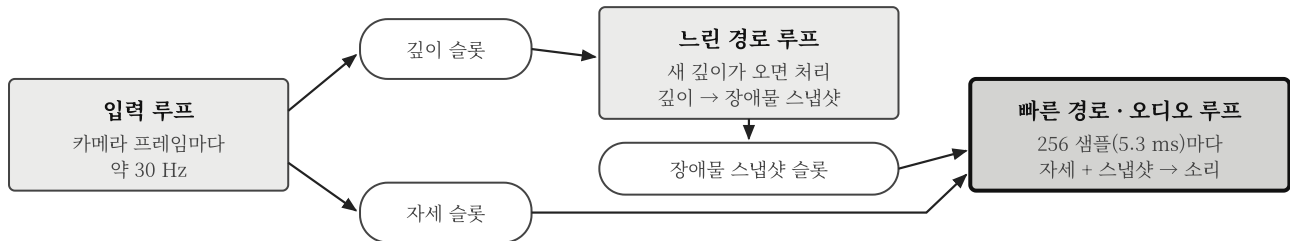


그림 6. 세 루프와 최신 값 슬롯. 둥근 상자가 슬롯이다. 쓰는 쪽은 슬롯의 값을 덮어쓰고, 읽는 쪽은 가장 최근 값만 가져간다. 오디오 루프는 장애물 정보가 늦어도 멈추지 않고, 지연 시간으로 안내 여부를 판단한다(그림 3).

3.3 설계 원칙: 안전을 위해 지키는 규칙

표 5. 설계 원칙과 구현. 모든 설정 조합에서 유지하며, 합성 장면 자동 시험으로 확인한다.

원칙	이유	구현
과거 값만 쓴다	미래 프레임을 기다리면 안내가 늦어진다	보간하지 않고, 깊이 영상은 그 영상을 찍은 시각의 자세로 놓는다
최신 값 교체	밀린 정보를 늦게 안내하지 않는다	스레드 사이 슬롯(그림 6)
오래된 정보로 안내하지 않는다	이미 지나친 장애물 위치를 알리면 위험하다	지연 시간 300 ms 초과 시 장애물 소리를 끄고 확인 불가 알림
무음 ≠ 장애물 없음	침묵을 안전으로 오해하지 않게 한다	믿을 수 없을 때는 알림음·진동을 반복(그림 3), 기록에도 구분해 남긴다
시야 밖은 지우지 않는다	세로로 든 폰의 시야는 좌우 약 $\pm 20^\circ$ 로 좁다	카메라가 보지 않는 칸은 점수를 깎지 않고 기억한다
수치는 설정에서	코드를 바꾸지 않고 조정·비교한다	모든 상수를 <i>default.json</i> 에서 읽고 시작할 때 검증한다

3.4 개발·검증 구조

보행 실험은 매번 같은 조건을 만들기 어렵다. 그래서 앱으로 한 번 녹화한 세션을 PC에서 **코어의 같은 코드로** 몇 번이고 다시 재생해, 설정을 바꿔 가며 같은 장면에서 비교한다. 재생 결과는 줄자로 잰 정답과 비교해 지표로 만들고, 평가 GUI에서 영상·지도·소리를 같은 시각에 맞춰 보고 듣는다.

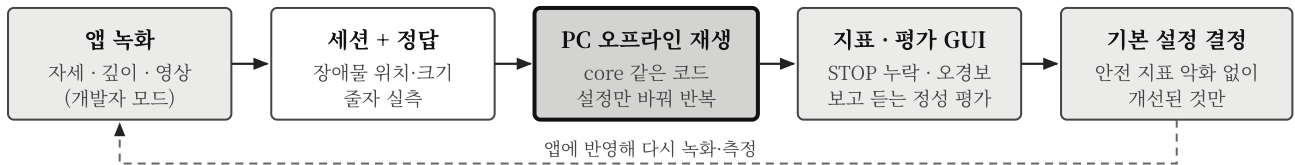


그림 7. 개발-검증 순환. 실제 녹화와 별도로, 바닥·벽·상자를 정확히 아는 합성 장면을 만들어 “STOP을 놓치지 않는다” 같은 안전 조건을 자동 시험으로 매번 확인한다.

3.5 성능 개선 단계에서 기본값에 반영된 것

MVP 이후 성능 개선 단계에서는 오차가 어디서 나는지 녹화로 나눠 재고(M12), 복셀 지도와 그 뒤 단계를 개선했다(M13). 여러 방법을 시험했지만, 지도·추적에서는 **조정에 쓰지 않은 확인용 녹화에서도 STOP 누락이 생기지 않고 계산 부담이 거의 없는** 세 가지만 기본값으로 했고, 음향은 팀원이 설계한 거리·높이 분리 매핑을 기본값으로 했다. 나머지는 꺼진 설정으로만 남겨 이 문서의 플로우에 넣지 않았다.

표 6. 기본값에 반영된 개선 (그림 4의 ★).

개선	문제	바꾼 것
바닥 후보 하한	끝 벽 앞에서 바닥이 안 보이면 벽 면을 바닥으로 잡아, 걷는 중 바닥 높이가 약 1 m 올라감	폰보다 0.8 m 이상 아래인 점만 바닥 후보로 쓴다
거리 가중 표	2 m보다 먼 깊이는 오차가 커서, 잘못 찍힌 먼 점이 장애물 칸으로 쌓임	먼 관측은 $(2/\text{거리})^2$ 만큼 약하게 센다
겹침 연결 추적	같은 장애물인데 중심이 튀어 추적 번호가 바뀌고, 소리가 새 장애물처럼 다시 시작됨	위에서 본 상자가 절반 넘게 겹치면 같은 장애물로 잇는다
거리·높이 음향 매핑	위험도 하나로 음높이·음량을 함께 바꿔 거리와 높이를 구분하기 어려움	거리 → 음량, 높이 → 음높이로 나누고, 동시에 음원 2개·STOP 우선

표 7. 지도·추적 기본값 변경 전후. 같은 복도에서 녹화한 세션을 이전 설정과 지금 설정으로 각각 재생해 비교했다. 장면마다 2~3번 녹화했으며 비율·거리는 그 중앙값, 횟수는 합이다.

지표 (단위)	장면	이전	지금	변화
STOP 누락 (건)	모든 장면	0	0	같음 (놓친 STOP 없음)
잘못된 경고 비율 (%) 올린 경고 중 실제 장애물과 맞지 않는 것	빈 복도	78%	46%	32%p 감소
	왕복 보행	50%	16%	34%p 감소
	벽에 붙은 캐리어	8%	1%	7%p 감소
추적 번호 바뀔 (회) 같은 장애물의 소리가 새로 시작된 횟수	복도 중앙 캐리어	14회	5회	9회 감소
	벽에 붙은 캐리어	8회	1회	7회 감소
물체-벽 합쳐짐 비율 (%) 장애물이 뒤 벽까지 한 덩어리로 잡힌 시간 비율	벽에 붙은 캐리어	60%	36%	24%p 감소
	빈 복도(끝 문)	90%	72%	18%p 감소
	복도 중앙 캐리어	51%	58%	7%p 증가
첫 STOP 거리 (m) 1.0 m보다 멀면 늦지 않음	빈 복도(끝 문)	1.26 m	1.06 m	0.20 m 늦어짐 (기준 안)
	복도 중앙 캐리어	1.07 m	1.16 m	0.09 m 빨라짐

잘못된 경고와 추적 번호 바뀔은 크게 줄었다. 반면 물체와 뒤 벽의 합쳐짐은 장면에 따라 줄거나 조금 늘었고(복도 중앙 캐리어), 물체 뒤 배경이 앞으로 당겨 보이는 ARCore 깊이 자체의 오차는 남아 있다. 이를 줄이는 깊이 영상 처리는 기기 발열 측정 뒤 적용 여부를 정한다(4절).

4 확장 계획

지금 구현은 제안서의 핵심 기능 중 “위치와 돌출부”를 공간음향으로 안내하는 부분이다. 남은 기능은 같은 구조 위에 모듈을 더하는 방식으로 넓힌다.

표 8. 확장 계획 (미구현).

확장	내용	구조에서 들어갈 곳
여러 물체·여러 음원	가까이 있는 물체를 따로 인식하고 개수·우선순위에 맞춰 동시에 들려줌	tracking(물체 단위 지도·추적), audio(여러 음원 음원화)
신호·공간 의미의 음성 안내	보행자 신호등, 통행 금지·공사 안내를 짧은 말로 알림 (제안서 “신호·공간 의미”)	RGB 영상 인식 모듈을 새로 두고, 음성 안내 중 공간음향 음량을 낮추는 우선순위를 guidance에 추가
깊이 정확도	ARCore 깊이의 계통 오차(물체 뒤 배경 당김, 먼 바닥 들림)를 줄임. 깊이 영상의 물체 경계 판정은 구현을 마쳤고, 기기 발열 측정 뒤 적용한다	느린 경로 앞단(깊이 영상 처리)
기기 부하	10분 이상 사용할 때 발열로 처리가 늦어지는 문제를 줄임	runtime(느린 경로 주기 제한)